FOR WHOEVER LOOKS AFTER THE APP

Fixing and changing Field Measure with AI

Field Measure 1.0.0 · 2026-10-10

For the person who looks after Field Measure. You don't need to read or write code. An AI coding agent does the coding; you describe what you want, check its work, and decide what ships.

The repository is set up for this. `AGENTS.md` gives the agent its rules. `docs/STATUS.md`, `docs/CHANGELOG.md` and `docs/DECISIONS.md` are its memory between sessions. About 1,800 automated tests check its work.

1. How a change reaches the crews

```
You describe the change
        |
The AI works on a separate BRANCH (a safe copy) and runs the tests
        |
The AI opens a PULL REQUEST: a summary of what it changed, for you to read
        |
GitHub re-runs every test on it (the "CI" check). Wait for the green check
        |
You press "Merge pull request"   ← this is the moment it ships
        |
About 5 minutes later the app updates itself (only if the tests pass on main)
        |
Surfaces show "Update ready". Try it on ONE Surface before telling the crew
        |
Wrong? Open the merged pull request and press "Revert", then merge that
```

Nothing reaches a Surface until you merge. A broken change never deploys: the app only updates after the tests pass.

2. What you need

- A **GitHub account** with access to your copy of the repository (see `README.md`, "Make your own copy").
- An **AI coding agent**. Pick one:

Agent	Install anything?	Notes
Claude Code on the web (claude.ai/code)	No	**Easiest.** Runs in the browser, connects to GitHub, runs the tests in the cloud and opens the pull request for you.
Claude Code (desktop app or terminal)	Yes	Works on your PC, on a copy of the repo you cloned.
Cursor, or VS Code with GitHub Copilot	Yes	Both read `AGENTS.md`. You'll push and open the pull request yourself (or ask the agent to).
OpenAI Codex	Depends	Reads `AGENTS.md`.

3. Route A: in the browser (nothing to install)

1. Go to **claude.ai/code** and sign in.
2. Connect GitHub when asked, and choose your **FieldMeasure** repository.
3. Start a session and paste the **"Get oriented"** prompt (section 5).
4. Then paste a **"Fix a bug"** or **"Change something"** prompt with your request.
5. When it says the pull request is open, follow section 6 ("Check the AI's work").

4. Route B: on your own PC (pull the repo down)

Use this if you prefer a desktop agent, or want to try a change on your PC before it ships.

One-time setup (about 20 minutes):

1. Install **Git for Windows**: https://git-scm.com/download/win (accept the defaults).
2. Install **Node.js 24 LTS**: https://nodejs.org/ (accept the defaults).
3. Optional, if you'd rather click than type: **GitHub Desktop**, https://desktop.github.com/.
4. Install your agent: Claude Code, Cursor, or VS Code with Copilot.

Get the code onto your PC. Use **Command Prompt**, not PowerShell:

```
cd %USERPROFILE%\Documents
git clone https://github.com/<your-github-name>/FieldMeasure.git
cd FieldMeasure
npm ci
npx playwright install chromium
```

With GitHub Desktop instead: **File** → **Clone repository** → **FieldMeasure** → **Clone**. Then open Command Prompt in that folder and run the last two lines.

Open it in your agent. Open the `FieldMeasure` folder in Claude Code, Cursor or VS Code, then paste the "Get oriented" prompt. The agent reads `AGENTS.md` by itself.

Look at the app on your PC:

```
npm run build
npm run preview
```

Then open http://localhost:4173 in Edge. Press **Ctrl+C** in the window to stop it. **Never use** `npm run dev`. It shows a broken, unstyled page on purpose.

Before each new piece of work, get the latest code: `git checkout main` then `git pull` (or **Fetch origin** → **Pull** in GitHub Desktop).

Send the change to GitHub. Ask the agent: *"Commit this on a new branch, push it, and open a pull request."* Or in GitHub Desktop: **Commit**, then **Push origin**, then **Create Pull Request**.

5. Prompts to copy

Replace the parts in `<angle brackets>`.

Get oriented (start every session with this):

```
Read AGENTS.md, docs/STATUS.md and docs/CHANGELOG.md. Tell me in plain English, in under 15 lines,
where the app stands and anything that looks broken. Don't change anything yet.
```

Fix a bug:

```
A crew member reported a problem.
What they did: <steps, e.g. "opened the Kitchen project, tapped Dimension, tapped two points">
What they expected: <...>
What happened: <...>
Surface model and how often it happens: <...>
<Attach the screenshot.>

Follow AGENTS.md. First reproduce it and explain the cause in plain English. Then fix it with a
test that fails before the fix and passes after. Run all the checks and show me the numbers.
Update docs/DECISIONS.md, docs/CHANGELOG.md and docs/STATUS.md. Open a pull request that says, in
plain English, what changed, what I should try on a Surface, and anything you could not test.
```

Change something or add a feature:

```
I want: <the change, described the way you'd tell a person>.
Follow AGENTS.md: work on a new branch and keep the change as small as possible. Put any new
on-screen words in src/ui/strings.ts and keep the in-app Help and docs/USER-GUIDE.md in step.
Run the checks (tsc, vitest, build, playwright) and show me the results. Never delete, skip or
loosen a test. If what the user sees changed, run npm run clickthru and show me the screenshots.
Update docs/DECISIONS.md, docs/CHANGELOG.md and docs/STATUS.md, then open a pull request
explaining what changed and what to try on a Surface.
```

Change wording on screen:

```
Change the on-screen text "<old words>" to "<new words>". Follow AGENTS.md (the copy rules in
src/ui/strings.ts and the copy test). If it appears in Help, update docs/USER-GUIDE.md too.
```

Change the logo or brand:

```
Replace the VANGARDE branding with <new name>. The new logo files are attached. Swap the files in
public/branding/ (same file names, similar proportions) and the app icon in public/icons/.
Run the checks and show me before and after screenshots of the editor and an exported PDF.
```

The check on my pull request is red:

```
The CI check on pull request #<number> failed. Read the failure, explain the cause in plain
English, and fix the code (not the test). Never skip or loosen a test. Push the fix to the same
branch.
```

Undo a bad update quickly:

```
The last merge to main broke <what broke>. Revert that merge on a new branch and open a pull
request for the revert. Don't try to fix it forward yet. Then tell me the cause.
```

Monthly health check:

```
Run every check on main and tell me if anything is failing. List any dependencies with known
security problems (npm audit), but don't upgrade anything.
```

6. Check the AI's work (before you merge)

Open the pull request on GitHub and go through this list:

1. **Is the CI check green?** If it's red, don't merge. Use the "check is red" prompt.
2. **Does the summary make sense to you?** It should say in plain English what changed and what to try. If it doesn't, ask the agent to explain.
3. **Did it touch tests?** Open **Files changed**. New tests are good. A test that is deleted, skipped (`.skip` , `fixme`), or has its numbers loosened is a red flag. Ask why.
4. **Did it touch the high-stakes files?** These are `src/domain/` (measurement maths), `src/export/` (the PDF) and `src/fs/projectStore.ts` (saving). Changes there need a clear reason and a new test.
5. **Is it small?** One change per pull request. A huge pull request for a small request is a red flag.
6. **Merge it**, wait for the update, and **try it on one Surface** before telling the crew.

7. Red flags: stop and ask

- "I disabled / skipped / removed the failing test."
- It wants to add a new package (`npm install something`) without a clear reason.
- It used `npm run dev` to check the app.
- It wants to add a server, a login, cloud storage or tracking.
- It changed many unrelated files.
- It says something works "on a Surface". It can't test on a real Surface; only you can.

8. Undoing a change

On GitHub, open the merged pull request and press **Revert**. GitHub creates a new pull request that undoes it. Merge that, and about 5 minutes later the Surfaces get the previous version back (tap **Reload** on **Update ready**). Crews' project files are never touched by an update.

9. Where things are

You want	Look at
The rules the AI follows	`AGENTS.md`
What works and what's open	`docs/STATUS.md`
What changed recently	`docs/CHANGELOG.md`
Why something is the way it is	`docs/DECISIONS.md` (search for the feature)
How the crews use the app	`docs/USER-GUIDE.md` (same text as the in-app Help)

You want	Look at
The hands-on test checklist	`docs/FIELD-TEST.md`
Measurement maths	`src/domain/units.ts`
All on-screen words	`src/ui/strings.ts`
How it was originally built	`docs/archive/` (read-only history)

10. When you get stuck

Problem	Fix
`git` or `npm` "is not recognized"	Install Git / Node.js 24 (section 4), then open a **new** Command Prompt.
"running scripts is disabled on this system"	Use Command Prompt instead of PowerShell, or type `npm.cmd`.
Tests fail with "Executable doesn't exist"	Run `npx playwright install chromium` once.
Port 4173 is already in use	Close the other preview window, or restart the PC.
The pull request says "conflicts"	Ask the agent: *"Merge main into this branch and resolve the conflicts."*
The app didn't update after a merge	Check **Actions** on GitHub. Deploy only runs after CI passes. Re-run a failed run once.
You're not sure a change is safe	Don't merge. Ask the agent to explain the risk in plain English.